

event called `navigatingTo`. We have provided a handler to handle that event. Similarly, on the button we have tap event and we provide a handler to handle that event. Notice the syntax `{{ message }}` on the label. It is called as DataBinding expression. We have data bound text property of the label to message property of something. That something is an object which will be bound to this page at runtime. Similarly, the tap event of the button is bound using a data binding expression to `onTap` function.

Business Logic for Page:

It's time to define the logic which powers our main-page. In the app folder you will find `main-page.ts` file. Let's take a look at the code in `main-page.ts`:

```
import { EventData } from 'data/observable';
import { Page } from 'ui/page';
import { HelloWorldModel } from './main-view-model';

// Event handler for Page "navigatingTo"
// event attached in main-page.xml
export function navigatingTo(args: EventData) {
    // Get the event sender
    let page = <Page>args.object;
    page.bindingContext = new HelloWorldModel();
}
```

We define a function `navigatingTo` and we export it. Export just makes our function available to the outside world. Remember the `navigatingTo` event on the Page element in `main-page.xml` – this is the function which will handle that event. We get hold of the current page and set its binding context to `HelloWorldModel`. Think of `HelloWorldModel` as a business object which handles all the logic of the screen/page. `HelloWorldModel` is defined in `main-view-model.ts` file. Here is the code snippet: <http://dgit.in/HelloWNS>.

The `HelloWorldModel` class is self-explanatory. We have defined two variables namely `_counter` and `_message`. We also have a public property called `message`. We have a function called `onTap`. Whenever `onTap` is executed it decrements the counter variable and updates the message. Remember the data binding expression `{{ message }}` on label in `main-page.xml` – the text will be updated anytime the message changes.

Styling UI Components:

I love NativeScript because of its ability to let me style UI components with CSS rules. In NativeScript, we support a subset of CSS 3.0. The basic rules such as background color, color, font, border etc. are supported. For our current application we have written a global style rules in `app.css`. You can create page specific rules by creating `<page-name>.css`. NativeScript will first look for page specific CSS file and apply it, if not found it will use the application wide CSS styles defined in `app.css`. Here is our `app.css` style sheet file:

Build and Run the App:

In order to build the app, you will use the following command:

This command will create the Android APK package

To run your app, you will issue the following command:

```
.title {
    tns build android
    tns run android
    button {
        font-size: 42;
        horizontal-align: center;
    }
    .message {
        font-size: 20;
        color: #284848;
        horizontal-align: center;
        margin: 0 20;
        text-align: center;
    }
}
```

It will deploy the app on any device connected to your system. You can run on emulator by passing the attribute `-emulator`. And here is the output of our app: NativeScript App running on Android.

Conclusion:

With NativeScript you can develop native Android/iOS apps with just your JavaScript knowledge. What's more NativeScript is all about open standards – XML, JavaScript and CSS. Hope this article gave you an idea of what NativeScript is capable of. This is just the tip of the iceberg. Head over to www.nativescript.org to learn more. >

*Nodes of Interest



*NativeScript tutorial

>>The official NativeScript tutorial divided into several segments like getting started, core concepts etc.
<http://dgit.in/NSTutor>



*Building a native app

>>Another tutorial on building cross platform native apps using NativeScript from Telerik.
<http://dgit.in/NSAppDev>



*Windows 10 Runtime

>>An overview of the open sourced NativeScript runtime for the Universal Windows Platform (UWP).
<http://dgit.in/NSMSWin>